

CSCI 198 Project Proposal

Title: *Music Intelligence with Deep Learning*

Student: *Noah Wiley*

Department of Computer Science, California State University, Fresno

Supervisor: *David Ruby*

Problem Statement:

Music classification identifies genre, emotion, or instrumentation from audio and remains a core open problem in Music Information Retrieval (MIR). While CNNs, LSTMs, and Transformers have individually shown promise, there is limited comparative work in an undergraduate setting that unifies these architectures under a single framework. This project aims to design, train, and evaluate these models on publicly available music datasets, culminating in a usable demo tool.

Background & Related Work:

CNNs excel at capturing spatial patterns in mel-spectrograms, LSTMs model temporal audio dependencies, and Transformers leverage self-attention to analyze long-range musical structure. Prior work in MIR which includes research from groups such as the Affective Music AI (AMAI) Lab demonstrates the effectiveness of deep learning approaches for emotion recognition, tagging, and genre classification.

Dataset Identification:

This project will utilize three widely adopted music datasets: GTZAN, FMA Small, and MagnaTagATune. GTZAN is a benchmark dataset for single-label genre classification containing ten distinct genres. FMA Small provides an additional balanced single-label genre dataset consisting of eight genres and is well suited for cross-dataset benchmarking. MagnaTagATune offers a multi-label classification task with instrument, mood, genre annotations, enabling evaluation of deep learning models under multi-label conditions. Together, these datasets provide diversity in label structure, dataset size, and annotation density. The inclusion of both single-label and multi-label datasets allows for evaluation across different loss functions, such as softmax cross-entropy and binary cross entropy, and enables deeper analysis of model generalization.

Proposed Approach:

- Preprocess audio into mel-spectrograms and MFCCs
- Implement and train models across three families: CNN, LSTM, Transformer
- Evaluate using accuracy, F1 score, and confusion matrices, and qualitative error analysis.
- Build an interactive tool allowing users to upload audio and receive model predictions
- Publish all code on GitHub and produce a final research report

Expected Outcomes:

- A complete GitHub repository with training code, model checkpoints, and documentation.
- A research quality report comparing deep learning architectures for MIR
- Demo classification software for student use
- Contributions that support refinement of the undergraduate Deep Learning course and the development of a future Graduate Deep Learning course.

Required Deliverables:

- Final Report
- GitHub repository
- Working demo tool
- End-of-semester departmental presentation (oral/poster) as required

Department Presentation Requirement:

According to the CSCI 198 course description, students must present their results in one of the approved formats:

1. An oral presentation organized by the department
2. The CSCI Poster Showcase
3. The CSM Culminating Experience Showcase

The preferred order is (1) > (2) > (3). A less-preferred format may only be selected if compelling reasons are provided and approved by the project supervisor.

Weekly Timeline:

Week 1 - Set up repository, folder structure, and Python environment. Review dataset documentation and confirm preprocessing requirements. Write a short project plan outlining model goals and evaluation criteria.

Week 2 - Download GTZAN, MTG-Jamendo, and MagnaTagATune from Hugging Face. Inspect audio formats, sampling rates, and metadata. Remove corrupted or duplicate files. Create a unified label schema.

Week 3 - Implement preprocessing functions for mel-spectrograms, MFCCs, and log-mel features. Test consistency across all datasets. Benchmark preprocessing runtime to estimate training feasibility.

Week 4 - Design CNN architecture tailored to spectrogram inputs. Train a prototype model to ensure pipeline integrity. Record initial metrics and identify bottlenecks.

Week 5 - Apply augmentations like noise addition and time stretching. Tune hyperparameters using validation splits. Save final CNN version and evaluation metrics.

Week 6 - Transform spectrograms into temporal sequences for LSTM input. Implement the LSTM or BiLSTM model. Validate that sequential data format is correct.

Week 7 - Train LSTM model, monitor overfitting, adjust sequence length and hidden sizes. Compare preliminary performance with CNN baseline.

Week 8 - Implement a Transformer based architecture that integrates positional encodings and attention layers.

Week 9 - Train Transformer model using optimized preprocessing. Log GPU usage and training time. Evaluate performance against prior models.

Week 10 - Compile metrics like accuracy, F1, confusion matrices for all three models. Produce visualization such as ROC curves, training curves, and parameter comparisons.

Week 11 - Analyze error cases, misclassified samples, and genre/emotion overlaps. Explore which architectures handle which classes best. Begin drafting the results section.

Week 12 - Create a prototype of the demo app using Streamlit or Flask. Build UI for uploading short audio snippets. Integrate preprocessing + inference pipeline

Week 13 - Refine UI, add instructions, and ensure stable performance. Test with varied audio sources like phone recording, MP3 samples. Gather feedback from peers and friends.

Week 14 - Write Introduction, Related Work, and Methods sections. Insert figures and tables. Draft Results and Discussion.

Week 15 - Revise the entire report with supervisor comments. Clean up repository structure, add documentation, examples, and usage instructions. Finalize demo total.

Week 16 - Prepare required CSCI 198 oral/poster presentation . Deliver presentation. Submit report, GitHub link, software demo, and all final deliverables.

Justification for 3 Units

A 3-unit CSCI 198 project requires approximately 135 hours of work across the semester, and this project clearly meets that standard. The workload includes designing and comparing three major deep learning architectures, each requiring substantial training time, data preprocessing, debugging, and evaluation. Working with large audio datasets adds additional complexity in storage, feature extraction, and computation. The creation of a functional demo application

introduces a software engineering component beyond the research itself. Finally, the project requires a formal written report, a structured evaluation of results, and preparation for a department-approved presentation. Combined, these tasks constitute a full scale research and development effort appropriate for a 3-unit senior project.